Planta de ensamble de 5 robots: propuesta de sincronización

Manuel Enrique Rojas Villegas **Alexander Cifuentes Ortíz**

26 de Octubre, 2026

Borrador para revisión

Resumen

El *brain* (un nodo ROS 2) recibe de la GUI las cajas a ensamblar, asigna cada etapa a los 5 robots (uno por etapa) según los eventos que estos publican y devuelve a la GUI el avance, las alertas y el estado de la planta. Todo implementado en Python (`rclpy`), sobre una red local propia.

Índice

1 Decisiones propuestas **2**

2 Interfaz con la GUI **2**

3 Arquitectura **3**

4 Mensajes **3**

5 Flujo del coordinador **4**

6 Fallos y seguridad **4**

7 Contrato de integración **4**

7.1 Reglas comunes 6
7.2 Cada robot (grupos 1 a 5) 6
7.3 Robot 2: selección de medicamento 8
7.4 GUI 9
7.5 Cómo probar sin el resto del sistema 10

1 Decisiones propuestas

Tema	Propuesta	Razón
Comunicación	ROS 2: el brain y cada robot son nodos (`rclpy`) en la red del router	Tópicos y servicios con entrega confiable, orientados a eventos; tecnología ya decidida
Coordinador	Uno solo (el brain); habla con la GUI y con los 5 robots	Es lo definido; si cae, los robots pasan a parada segura
Confirmación	Sí, en dos niveles: QoS confiable de ROS 2 (transporte) y estado `ejecutando` (aplicación)	Garantiza que la orden llegó y que el robot la aceptó
Robot que no responde	Esperar, dar 3 intentos y, si no responde, avisar a la GUI y parar todo hasta que mande continuar	El fallo queda avisado y nada avanza sin la orden de la GUI (ver sección 6)
Parada de emergencia	Sí, global: la envía la GUI por un tópico ROS 2	Requisito de seguridad en una planta con robots
Lenguaje	Python en todos los nodos	Lo definido

Cuadro 1: Decisiones de diseño y arquitectura planteadas.

2 Interfaz con la GUI

Función	Interfaz ROS 2	Sentido	Contenido
Iniciar una caja	Servicio `/gui/iniciar_caja`	GUI → brain	`caja_id` y medicamento (`AA`, `AB` o `BB`); el brain responde si la acepta
Parada global	Tópico `/planta/estop`	GUI → brain y robots	Activar o liberar
Continuar tras un fallo	Servicio `/gui/continuar`	GUI → brain	Reanuda la planta
Avance de la caja	Tópico `/planta/progreso`	Brain → GUI	`caja_id`, etapa, porcentaje
Alertas	Tópico `/planta/alerta`	Brain → GUI	`robot_id`, causa del fallo
Estado de la planta	Tópico `/planta/estado`	Brain → GUI	Estado de cada robot y de la planta

Cuadro 2: Interfaces de comunicación con la GUI.

Los nombres y tipos de mensaje son una propuesta: hay que acordarlos con quien desarrolla la GUI.

3 Arquitectura

El brain conecta la GUI con los 5 robots.

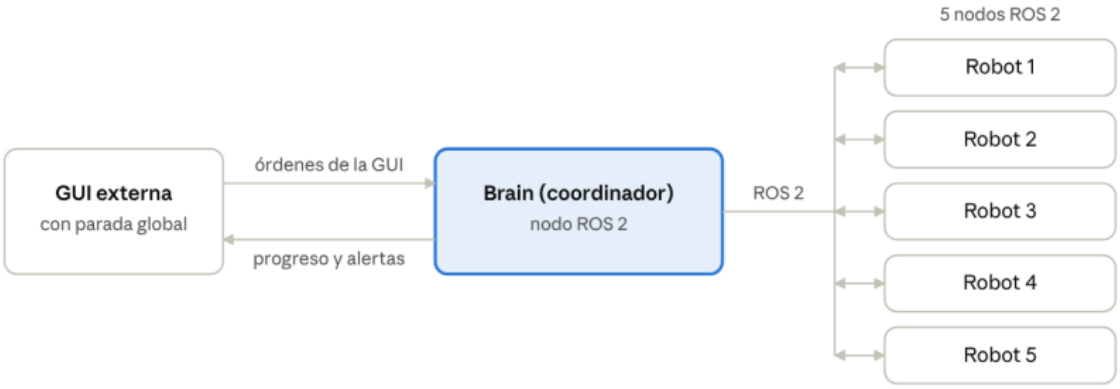


Figura 1: Arquitectura del sistema. GUI externa con parada global, nodo coordinador brain y 5 robots ROS 2.

La GUI manda órdenes al brain y recibe progreso y alertas; el brain habla con los 5 robots por ROS 2.

4 Mensajes

Tópico	Dirección	Contenido	QoS
`/robot_{id}/cmd`	Brain → robot	`caja_id`, `tarea`, `seq`	Confiable
`/robot_{id}/estado`	Robot → brain	`robot_id`, `estado`, `caja_id`, `seq`	Confiable
`/brain/latido`	Brain → robots	Latido periódico	Confiable (profundidad 1)
`/planta/estop`	GUI → brain y robots	Parada global	Confiable, transient local

Cuadro 3: Tópicos ROS 2 del sistema.

ID	Estado	Significado
0	`esperando`	Libre y listo para recibir tarea
1	`ejecutando`	Aceptó la tarea (hace de confirmación)
2	`finalizo`	Terminó su etapa de la caja
3	`error`	Falló; requiere atención

Cuadro 4: Estados definidos para los robots.

Mensajes propuestos (archivos `.msg`)

```
# RobotCmd.msg
uint8 robot_id
uint32 caja_id
string tarea
string medicamento # AA, AB o BB; solo lo usa el robot 2
uint32 seq
```

```
7
8  # RobotEstado.msg
9  uint8 robot_id
10 uint8 estado # 0 esperando, 1 ejecutando, 2 finalizo, 3 error
11 uint32 caja_id
12 uint32 seq
13
14 # Progreso.msg
15 uint32 caja_id
16 uint8 etapa # 1 a 5
17 uint8 porcentaje # 20, 40, 60, 80, 100
```

Listing 1: Definición de mensajes ROS 2 (.msg)

5 Flujo del coordinador

Regla: la caja k pasa al robot i cuando el robot $i-1$ finalizó la caja k y el robot i está esperando. Así puede haber varias cajas en la línea, cada una en una etapa distinta. Cada etapa terminada vale 20 % de la caja: el brain informa a la GUI el avance de la caja x (20, 40, 60, 80 y 100 %).

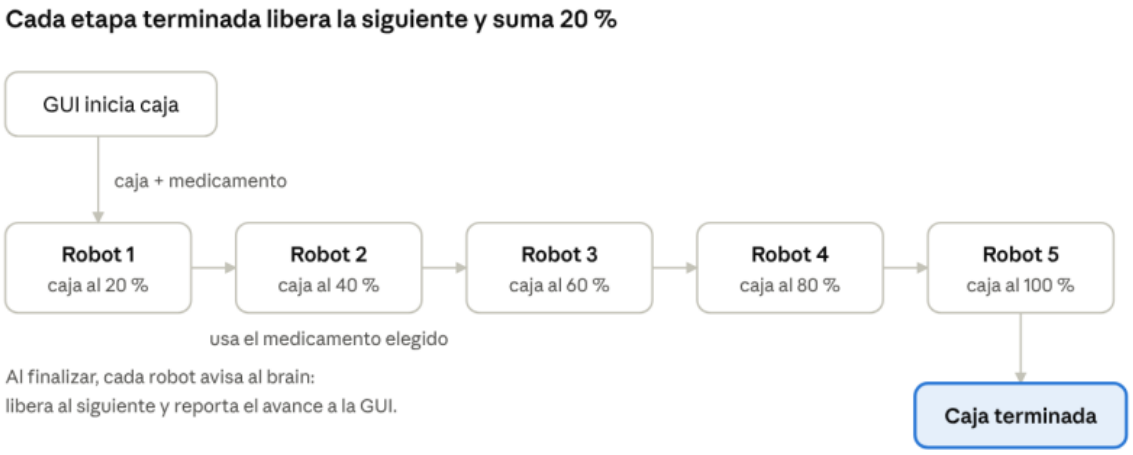


Figura 2: Flujo de 5 etapas, avance de 20 % en cada una. La GUI inicia la caja, que pasa por las 5 etapas en orden y suma 20 % en cada una.

Al finalizar, cada robot avisa al brain: libera al siguiente y reporta el avance a la GUI.

6 Fallos y seguridad

Un fallo detiene toda la planta hasta que la GUI diga continuar.

7 Contrato de integración

Esta sección fija qué publica y qué recibe cada parte, para que los grupos de cada robot y de la GUI integren sin ambigüedades. Los nombres y campos son los del nodo brain (`brain_node.py`).

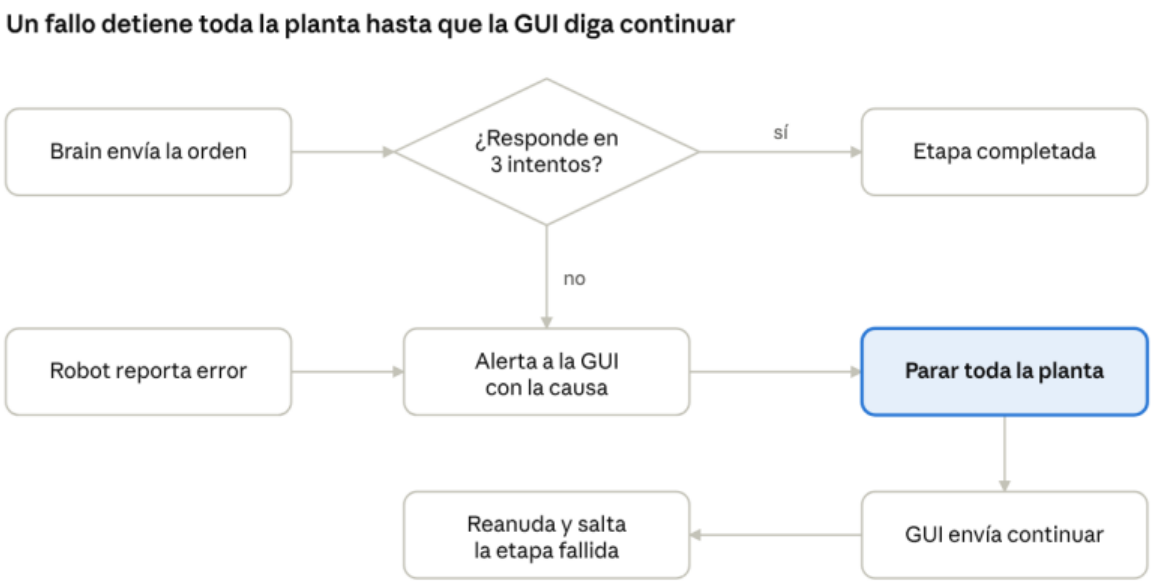


Figura 3: Gestión de fallos: 3 intentos, aviso y parada total. Sin respuesta tras 3 intentos, o con un error, el brain avisa a la GUI y para toda la planta hasta recibir continuar.

Situación	**Detección**	**Acción**
Robot no confirma la orden	3 intentos sin estado `ejecutando` (T_{ack} cada uno)	Alerta a la GUI con la causa y parada de toda la planta; al recibir continuar se salta esa etapa
Robot no termina a tiempo	Tiempo T_{tarea}	Alerta a la GUI y parada de toda la planta hasta continuar
Robot reporta error	Estado 3	Alerta a la GUI y parada de toda la planta hasta continuar
Robot se desconecta	Robot sin publicar estado por más de t_{offline} (3 s)	Alerta a la GUI y parada total si tenía trabajo (caja en proceso, orden pendiente o cola); si estaba libre, solo alerta
Cae el brain	Robots sin latido en T_{hb}	Parada segura de todos
Parada global	Tópico `/planta/estop` desde la GUI	Todos paran; se reanuda cuando la GUI manda continuar
Mensaje duplicado o viejo	Número `seq`	Se ignora

Cuadro 5: Matriz de detección y manejo de fallos.

Tema	Regla
Red e Interfaces	Todos los nodos en la red del router y con el mismo `ROS_DOMAIN_ID` (valor por definir). Todos usan el mismo paquete `planta_interfaces`: `RobotCmd`, `RobotEstado`, `Progreso`, `Alerta`, `EstadoPlanta` e `IniciarCaja`.
QoS	Confiable, `KEEP_LAST` con profundidad 10. Excepción: `/planta/estop` (confiable + transient local). El latido `/brain/latido` también es confiable, que conecta con suscriptores confiables y best effort.
Estados del robot	0 esperando, 1 ejecutando, 2 finalizo, 3 error.
seq	Número que genera el brain en cada orden; el robot lo repite igual en su respuesta.
Tiempos iniciales	Confirmar una orden en 2 s (hasta 3 intentos), terminar la etapa en 30 s, publicar el estado cada 1 s. Se ajustan midiendo cada robot.

Cuadro 6: Reglas comunes del contrato de integración.

7.1 Reglas comunes

7.2 Cada robot (grupos 1 a 5)

Los 5 robots son un nodo ROS 2 con el mismo contrato; solo cambia el número N (1 a 5).

Dirección	Tópico	Mensaje	Cuándo
Recibe	`/robot_N/cmd`	`RobotCmd`	Cuando el brain da una orden
Publica	`/robot_N/estado`	`RobotEstado`	Al cambiar de estado y repetido cada 1 s (sirve de latido)
Recibe	`/planta/estop`	`std_msgs/Bool`	`data: true` = parar de inmediato
Recibe	`/brain/latido`	`std_msgs/UInt32`	Cada 1 s; si no llega en T_{hb} (propuesta: 3 s), parada segura

Cuadro 7: Interfaces por robot.

Reglas importantes:

- No publicar estado 0 mientras tenga una caja en proceso: el brain le mandaría otra orden.
- Si algo falla, publicar estado 3 con la `caja_id`. No hace falta esperar respuesta: el brain avisa a la GUI y para la planta.
- Si no confirma una orden con estado 1 en 2 s, el brain la reenvía (hasta 3 intentos) y después declara fallo. Si deja de publicar su estado por más de 3 s, el brain lo da por desconectado.

Ejemplo para el robot 3 (la caja 17 llega a su etapa):

```
# /robot_3/cmd (brain -> robot 3)
robot_id: 3
caja_id: 17
tarea: 'T3'
medicamento: ''
seq: 41

# /robot_3/estado (robot 3 -> brain)
# 1) Al aceptar la orden (de inmediato)
robot_id: 3
```

tarea	Qué hace el robot	Qué publica
`T1` a `T5` (la de su robot)	Inicia la etapa de esa caja	De inmediato, estado 1 con la misma `caja_id` y `seq`. Al terminar, estado 2. Cuando quede libre, estado 0
Misma tarea con la misma `seq` que la actual	Es un reintento: no repite la tarea	Vuelve a publicar estado 1
`PAUSAR`	Se detiene sin perder la tarea	Sigue publicando su estado actual
`REANUDAR`	Retoma donde quedó	Su estado actual
`ABORTAR`	Cancela la tarea y pasa a posición segura	Estado 0 cuando esté listo

Cuadro 8: Comportamiento según la tarea recibida.

```
estado: 1
caja_id: 17
seq: 41

# 2) Al terminar su etapa
robot_id: 3
estado: 2
caja_id: 17
seq: 41

# 3) Cuando queda libre para otra caja
robot_id: 3
estado: 0
caja_id: 0
seq: 41
```

Listing 2: Ejemplo de flujo de mensajes para Robot 3

Esqueleto mínimo de un robot en Python

```python
import rclpy
from rclpy.node import Node
from rclpy.qos import HistoryPolicy, QoSProfile, ReliabilityPolicy
from planta_interfaces.msg import RobotCmd, RobotEstado

ROBOT_ID = 3  # cambiar según el robot (1 a 5)
ESPERANDO, EJECUTANDO, FINALIZO, ERROR = 0, 1, 2, 3

class RobotNode(Node):
    def __init__(self):
        super().__init__(f'robot_{ROBOT_ID}')
        qos = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,
            history=HistoryPolicy.KEEP_LAST,
            depth=10
        )
        self.pub = self.create_publisher(RobotEstado, f'/robot_{ROBOT_ID}/estado', qos)
        self.create_subscription(RobotCmd, f'/robot_{ROBOT_ID}/cmd', self.on_cmd, qos)
        self.estado, self.caja_id, self.seq = ESPERANDO, 0, 0
```

```python
        self.create_timer(1.0, self.publicar) # repetir el estado cada 1 s

    def publicar(self):
        m = RobotEstado()
        m.robot_id, m.estado = ROBOT_ID, self.estado
        m.caja_id, m.seq = self.caja_id, self.seq
        self.pub.publish(m)

    def cambiar_estado(self, estado, caja_id=None):
        self.estado = estado
        if caja_id is not None:
            self.caja_id = caja_id
        self.publicar() # publicar de inmediato al cambiar

    def on_cmd(self, cmd):
        if cmd.tarea == f'T{ROBOT_ID}':
            if cmd.seq == self.seq: # reintento: solo reconfirmar
                self.publicar()
                return
            self.seq = cmd.seq
            self.cambiar_estado(EJECUTANDO, cmd.caja_id)
            # TODO: iniciar la etapa real (el robot 2 usa cmd.medicamento)
        elif cmd.tarea == 'PAUSAR':
            pass # TODO: detener el movimiento sin perder la tarea
        elif cmd.tarea == 'REANUDAR':
            pass # TODO: retomar la tarea
        elif cmd.tarea == 'ABORTAR':
            # TODO: cancelar la tarea y dejar el robot en posición segura
            self.cambiar_estado(ESPERANDO, 0)

    def etapa_terminada(self): # llamar cuando el robot acabe su etapa
        self.cambiar_estado(FINALIZO)
        # TODO: esperar a quedar libre y entonces:
        self.cambiar_estado(ESPERANDO, 0)

    def etapa_fallida(self): # llamar si algo sale mal
        self.cambiar_estado(ERROR)
```

Listing 3: Esqueleto de implementación para un robot en rclpy

7.3 Robot 2: selección de medicamento

El robot 2 es el único que recibe un dato extra: el medicamento. El operador lo elige en la GUI al iniciar la caja; el robot 2 no le pide nada a la GUI.

Aspecto	Regla
Campo	`medicamento` en `RobotCmd`, con valor `AA`, `AB` o `BB`.
Quién lo recibe	Solo el robot 2. Los demás reciben medicamento vacío y lo ignoran.
Valor no válido	Si llega vacío o con otro valor, publicar estado 3 sin iniciar la etapa (el brain ya valida, es solo una protección).
Respuesta	Igual que en los demás robots: estado 1 al aceptar, 2 al terminar, 0 al quedar libre.

Cuadro 9: Especificación para el Robot 2.

Ejemplo (caja 17 con medicamento AB):

```
# /robot_2/cmd (brain -> robot 2)
robot_id: 2
```

```
caja_id: 17
tarea: 'T2'
medicamento: 'AB'
seq: 42

# /robot_2/estado (robot 2 -> brain), al aceptar la orden
robot_id: 2
estado: 1
caja_id: 17
seq: 42
```

Listing 4: Intercambio de mensajes para Robot 2

7.4 GUI

Dirección	Interfaz	Contenido
GUI → brain	Servicio `/gui/iniciar_caja` (`IniciarCaja`)	`caja_id` (único y mayor que 0) y `medicamento` (`AA`, `AB` o `BB`)
GUI → brain y robots	Tópico `/planta/estop` (`Bool`, confiable + transient local)	`true` para parar, `false` para liberar
GUI → brain	Servicio `/gui/continuar` (`std_srvs/Trigger`)	Reanuda tras un fallo o una parada
Brain → GUI	Tópico `/planta/progreso` (`Progreso`)	Una vez por cada etapa completada
Brain → GUI	Tópico `/planta/alerta` (`Alerta`)	Cuando ocurre un fallo
Brain → GUI	Tópico `/planta/estado` (`EstadoPlanta`)	Cada 1 s

Cuadro 10: Contrato de comunicación con la GUI.

Reglas para la GUI:

- Iniciar cajas solo cuando el estado de la planta sea `OPERANDO`. Si no, el brain responde `aceptada: false` con el motivo, y la GUI debe mostrarlo.
- El avance de cada caja es el porcentaje de `Progreso`: 20, 40, 60, 80 y 100. Una caja con una etapa saltada nunca llega a 100 %.
- Ante una alerta, mostrar `robot_id`, `caja_id` y causa, y habilitar el botón Continuar. Una alerta con `robot_id` 0 avisa que una caja terminó incompleta.
- Tras una parada global, primero liberar (`false`) y después continuar. Si la parada sigue activa, el brain rechaza el continuar.
- Suscribirse a `/planta/estop` con QoS *transient local* para no perder el estado de la parada.

Ejemplos de lo que manda y recibe la GUI:

```
# Respuesta a /gui/iniciar_caja
aceptada: true
motivo: ''

# o, si la planta esta parada:
aceptada: false
motivo: 'La planta esta en FALLO; no se aceptan cajas'

# /planta/progreso (la caja 17 termino la etapa del robot 2)
caja_id: 17
etapa: 2
porcentaje: 40

# /planta/alerta (el robot 4 reporto un error)
robot_id: 4
caja_id: 17
causa: 'El robot 4 reporto un error'

# /planta/estado (cada 1 s)
estado: 'FALLO' # OPERANDO, FALLO, PARADA_GLOBAL
estados_robots: [0, 0, 1, 3, 0]
cajas_robots: [0, 0, 18, 17, 0]
cajas_en_cola: [0, 0, 0, 0, 0]
```

Listing 5: Ejemplos de mensajes de la GUI

7.5 Cómo probar sin el resto del sistema

Grupo de un robot: simular al brain y mirar la respuesta.

```
ros2 topic pub --once /robot_3/cmd planta_interfaces/msg/RobotCmd "{robot_id: 3,
    caja_id: 17, tarea: 'T3', medicamento: '', seq: 41}"
ros2 topic echo /robot_3/estado
```

Grupo de la GUI: iniciar una caja, parar, liberar y continuar.

```
ros2 service call /gui/iniciar_caja planta_interfaces/srv/IniciarCaja "{caja_id: 17,
     medicamento: 'AB'}"
ros2 topic pub --once --qos-reliability reliable --qos-durability transient_local /
    planta/estop std_msgs/msg/Bool "{data: true}"
ros2 topic pub --once --qos-reliability reliable --qos-durability transient_local /
    planta/estop std_msgs/msg/Bool "{data: false}"
ros2 service call /gui/continuar std_srvs/srv/Trigger "{}"
ros2 topic echo /planta/progreso
```